11、gpio输出

全志V3S嵌入式驱动开发（gpio输出）

原创

嵌入式-老费 已于 2023-08-29 14:50:52 修改

Open: 8a923f94c34ac151897c6be747452d7c_MD5.png

阅读量1.7k Open: 196806fd3bdd4f04e1a92a7697cf0354_MD5.png

收藏 3

Open: 95eb4f436541fdf1b2e1af49844ad0c3_MD5.png

点赞数

CC 4.0 BY-SA版权

分类专栏： 基于全志soc的linux嵌入式开发 文章标签： 驱动开发

Open: 135c7190ea3ccdc82fa1f0c0a41d08f0_MD5.png

基于全志soc的linux嵌入式开发专栏收录该内容](https://blog.csdn.net/feixiaoxing/category_12270221.html "基于全志soc的linux嵌入式开发")

73 篇文章¥49.90¥99.00

已订阅8折续费

[

Open: d9b05f322fadabcd81dc6c5654189e59_MD5.png

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益Open: ddf4d655e9aa3806864ef10c7c4a060b_MD5.png

](https://www.csdn.net/vip?utm_source=bkzl_cjhy_ckqy)

Open: 1e4395309f4514fed3e1f5e89b2a25e6_MD5.png

本文介绍了如何将全志V3S芯片的SPI接口复用为GPIO，并详细阐述了修改DTS文件、重启开发板、查看端口使用情况以及创建通道进行设备外设驱动的过程。通过将SPI接口的CS引脚作为GPIO使用，展示了GPIO通道的配置和电平设置方法。

摘要生成于 C知道 ，由 DeepSeek-R1 满血版支持， 前往体验 >

【 声明：版权所有，欢迎转载，请勿用于商业用途。 联系信箱：feixiaoxing@163.com】

不管是对mcu，还是对soc来说，gpio肯定是越多越好。但是一个芯片上，它的引脚总是有限的，特别对于非BGA的soc来说，芯片又要做的小，引脚又要多，这几乎是不可能的。因此，在芯片领域，很多引脚功能都是复用的，至于选用哪一个功能完全看客户自己的选择，很多时候只能2选1，或者3选1。

对于v3s也是一样的，从全志的芯片手册，也就是Allwinner_V3s_Datasheet_V1.0.pdf中的第53页，我们看到v3s中很多的功能也是复用的，

Open: 957bc4e63e63172456089c15bc462f9f_MD5.png

3.2. GPIO Multiplex Functions

The following table provides a description of the V3s GPIO multiplexing functions.

Table 3-2. Multiplexing Functions

Pin Name	Default Function	IO Type	Default IO State	Default Pull-up/ down	Function 2	Function3	Function 4	Function 5	Function 6
PB0	GPIO	I/O	DIS	Z	UART2_TX	-	-	-	PB_EINT0
PB1		I/O	DIS	Z	UART2_RX	-	-	-	PB_EINT1
PB2		I/O	DIS	Z	UART2_RTS	-	-	-	PB_EINT2
PB3		I/O	DIS	Z	UART2_CTS	-	-	-	PB_EINT3
PB4		I/O	DIS	Z	PWM0	-	-	-	PB_EINT4
PB5		I/O	DIS	Z	PWM1	-	-	-	PB_EINT5
PB6		I/O	DIS	Z	TWI0_SCK	-	-	-	PB_EINT6
PB7		I/O	DIS	Z	TWI0_SDA	-	-	-	PB_EINT7
PB8		I/O	DIS	Z	TWI1_SCK	UART0_TX	-	-	PB_EINT8
PB9		I/O	DIS	Z	TWI1_SDA	UART0_RX	-	-	PB_EINT9
PC0	GPIO	I/O	DIS	Z	SDC2_CLK	SPI0_MISO	-	-	-
PC1		I/O	DIS	Z	SDC2_CMD	SPI0_CLK	-	-	-
PC2		I/O	DIS	Z	SDC2_RST	SPI0_CS	-	-	-
PC3		I/O	DIS	Z	SDC2_D0	SPI0_MOSI	-		
PE0		I/O	DIS	Z	CSI_PCLK	LCD_CLK	-	-	-
PE1		I/O	DIS	Z	CSI_MCLK	LCD_DE	-	-	-
PE2		I/O	DIS	Z	CSI_HSYNC	LCD_HSYNC	-	-	-
PE3		I/O	DIS	Z	CSI_VSYNC	LCD_VSYNC	-	-	-
PE4		I/O	DIS	Z	CSI_D0	LCD_D2	-	-	-
PE5		I/O	DIS	Z	CSI_D1	LCD_D3	-	-	-
PE6		I/O	DIS	Z	CSI_D2	LCD_D4	-	-	-

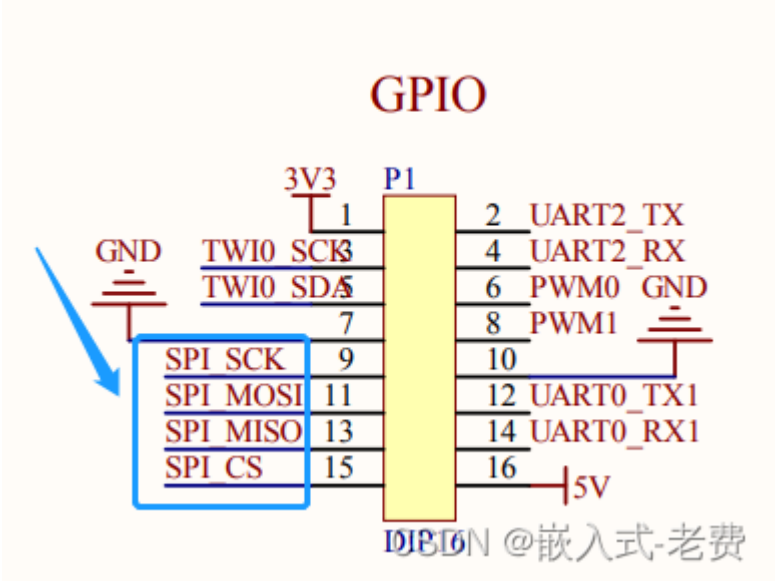
confidential

CSDN @嵌入式-老费

所以，大家如果在荔枝派中看到，给客户的引脚只有那么几个，这其实是误解。有很多的pin，如果不使用，完全可以拿过来直接当成gpio使用。

1、使用spi接口当成gpio口

Open: 5d83300a20f892a865d4d90f57986db8_MD5.png



这张图我们已经使用过很多次。图中有一个spi接口，之前主要是用作norflash访问使用的。现在因为所有系统都保存在sd卡里面，因此完全可以用这个当成gpio使用。

2、修改sun8i-v3s.dtsi文件

首先注释掉之前spi0_pins这个部分，

```
        /*spi0_pins: spi0 {
            pins = "PC0", "PC1", "PC2", "PC3";
            function = "spi0";
        };*/
```

接着注释掉spi0，

```
    /*spi0: spi@1c68000 {
        compatible = "allwinner,sun8i-h3-spi";
        reg = <0x01c68000 0x1000>;
        interrupts = <GIC_SPI 65 IRQ_TYPE_LEVEL_HIGH>;
        clocks = <&ccu CLK_BUS_SPI0>, <&ccu CLK_SPI0>;
        clock-names = "ahb", "mod";
        pinctrl-names = "default";
        pinctrl-0 = <&spi0_pins>;
        resets = <&ccu RST_BUS_SPI0>;
        status = "disabled";
        #address-cells = <1>;
        #size-cells = <0>;
    };*/
```

注释掉这两部分呢，重新编译成sun8i-v3s-licheepi-zero-dock.dtb文件就可以了。 **细心的同学也许会看到sun8i-v3s-licheepi-zero.dts和sun8i-v3s-licheepi-zero-dock.dts这两个文件中均有leds的配置，是不是status设置为okay就好了？要注意它们的状态都是写死的，后期不能通过命令和配置的方法来解决，虽然启动后也可以在/sys/kernel/debug/gpio下面看到映射关系，这个需要注意下。**

3、重启开发板

重启开发板之后，首先需要查看一下端口使用情况，没有debug信息，先要mount debugfs系统，

```
mount -t debugfs debugfs /sys/kernel/debug
```

加载好了之后，就可以看看端口的使用情况了，

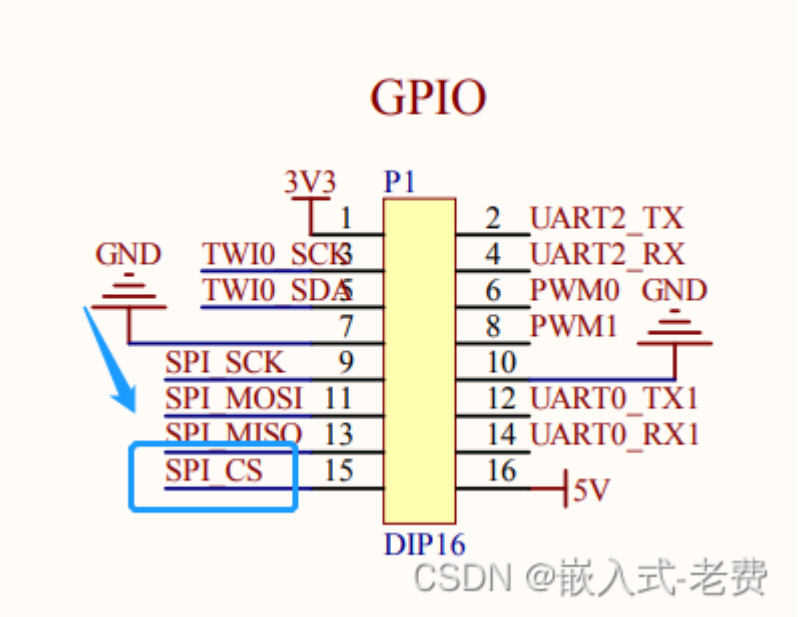
```
# cat /sys/kernel/debug/gpio
gpiochip0: GPIOs 0-223, parent: platform/1c20800.pinctrl,
1c20800.pinctrl:
 gpio-166 (                    |usb0_id_det         ) in  lo
```

看上去gpio情况还算正常。

4、创建通道，开始设备外设

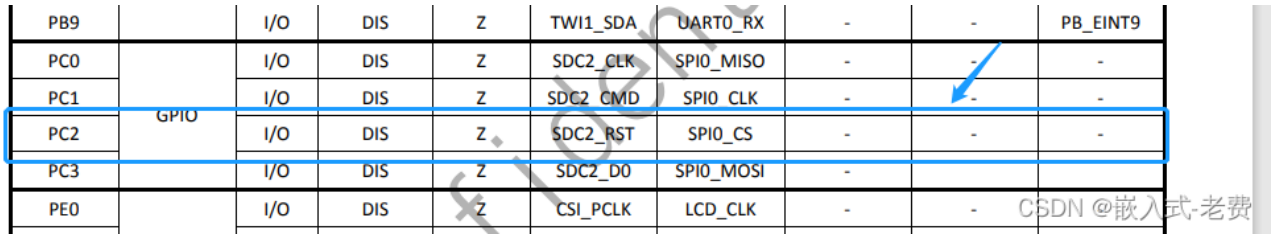
看过上面一篇文章的同学，对于/sys/class/pwm里面的export不会陌生。但是pwm只有两个，分别是0和1，gpio这么多，我们怎么把这些pin和通道bind在一起呢？其实这里面是有规律的。首先我们找到一个pin，但不知道它的序号是多少，那可以先找到名称，比如SPI_CS,

Open: ddcd20b5325421224cc8493cd953972a_MD5.png



接着看Allwinner_V3s_Datasheet_V1.0.pdf中的第54页，获取引脚名称，

Open: ba783f5049ad594795b0362747a8fa67_MD5.png



PB9		I/O	DIS	Z	TWI1_SDA	UART0_RX	-	-	PB_EINT9
PC0	GPIO	I/O	DIS	Z	SDC2_CLK	SPI0_MISO	-	-	-
PC1		I/O	DIS	Z	SDC2_CMD	SPI0_CLK	-	-	-
PC2		I/O	DIS	Z	SDC2_RST	SPI0_CS	-	-	-
PC3		I/O	DIS	Z	SDC2_D0	SPI0_MOSI	-		
PE0		I/O	DIS	Z	CSI_PCLK	LCD_CLK	-	-	

CSDN @嵌入式-老费

找到了这个信号叫PC2，下面就好办了。 **所有的端口一般都是channel = 32*x+y来实现的。PA、PB、PC...，这些代表x，分别是0、1、2...。而PC2中的2就代表y，如**

果是PB9，那么y就是9。 所以对于PC2来说，channel = 32 *2+ 2，也就是66，就是这么简单。那么，刚才说的PB9呢，它的channel = 32* 1 + 9，应该就是41。

说了这么多，下面开始实验，

```
echo 66 > /sys/class/gpio/export
echo out > /sys/class/gpio/gpio66/direction
echo 1 > /sys/class/gpio/gpio66/value
echo 0 > /sys/class/gpio/gpio66/value
```

四条命令依次解释下，第一条创建channel 66。第二条呢，设定chanel 66的方向为输出。第三条，设置高电平，与此相对的，第四条就是设置低电平。

为了验证设置的电平是不是正确，一个靠谱的办法就是在spi_cs处于高电平和低电平的时候都测量下，这样就知道电压有没有设置对了。

Open: c5f973bd55fedde4fbe44f648f3b016b_MD5.png